

Trabajo Práctico N°4: Representación del Conocimiento y Razonamiento Lógico

Integrantes del Equipo

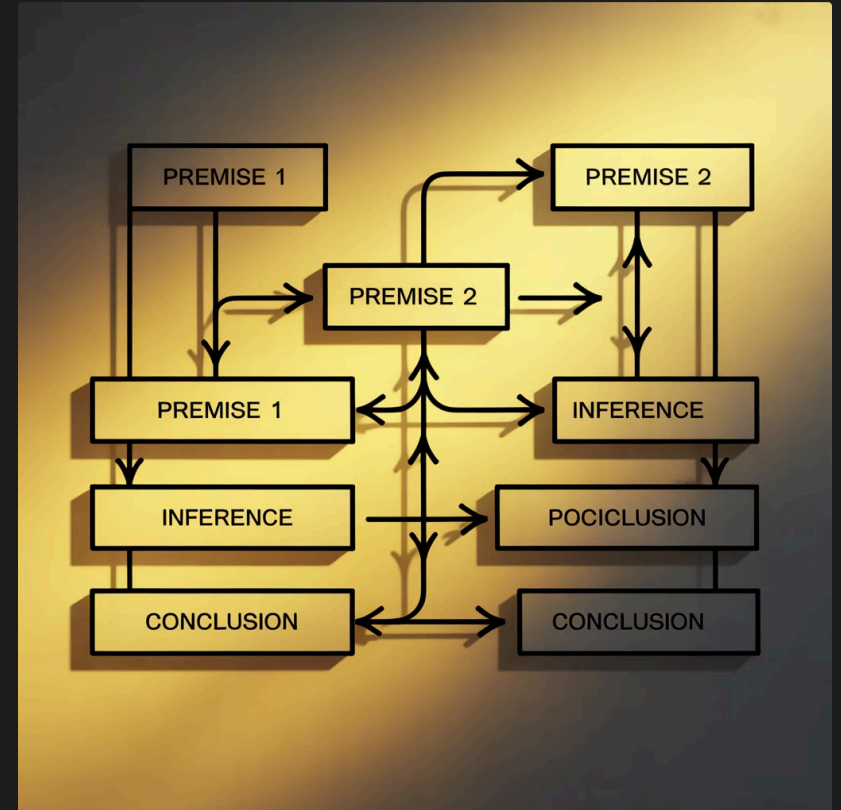
- Alarcón Ana
- Dujanoff Franco
- Salinas Eduardo
- Tomazelli Máximo



Punto 1: ¿Qué es una Inferencia?

La **inferencia** es el proceso cognitivo mediante el cual un agente inteligente llega a nuevas conclusiones a partir del conocimiento que ya posee. Este mecanismo fundamental permite expandir la base de conocimientos sin requerir información adicional del entorno.

La inferencia transforma el conocimiento implícito en explícito, permitiendo que los agentes razonen sobre situaciones complejas.



Este proceso es esencial para la toma de decisiones inteligentes y la resolución de problemas en entornos dinámicos e inciertos.

Punto 2: Verificación de Modelos en Bases de Conocimiento

Para verificar que una sentencia se infiere correctamente de la base de conocimiento, debemos comprobar si es **verdadera en todos los modelos** donde la base de conocimientos también lo es.

01

Identificación de Modelos Válidos

Se determinan todos los modelos posibles donde la base de conocimientos es consistente y verdadera.

02

Evaluación de la Sentencia

Se verifica si la sentencia objetivo es verdadera en cada uno de estos modelos válidos.

03

Confirmación de Consecuencia Lógica

Si la sentencia es verdadera en todos los modelos válidos, entonces es una consecuencia lógica de la base de conocimientos.

Punto 3: Análisis de la Base de Conocimientos

$R1 : b \wedge c \rightarrow a$

$R2 : d \wedge e \rightarrow b$

$R3 : g \wedge e \rightarrow b$

$R4 : e \rightarrow c$

$R5 : d$

$R6 : e$

$R7 : a \wedge g \rightarrow f$

Reglas Condicionales

El sistema opera con reglas del tipo "Si... entonces..." que establecen relaciones causales entre diferentes proposiciones lógicas.

Hechos Base

Conjunto de proposiciones que se consideran verdaderas desde el inicio y sirven como punto de partida para el razonamiento.

Meta Objetivo

La proposición que deseamos demostrar o verificar utilizando las reglas y hechos disponibles en la base de conocimientos.

Punto 3.1: Encadenamiento Hacia Adelante

El **encadenamiento hacia adelante** es un método de inferencia que parte de los hechos conocidos y aplica progresivamente las reglas disponibles hasta alcanzar la meta deseada.

Proceso de Razonamiento:

1

Hechos Iniciales

R5 y R6 establecen que **d** y **e** son verdaderos

2

Aplicación de R2

Con **d** y **e** verdaderos, inferimos que **b** es verdadero

3

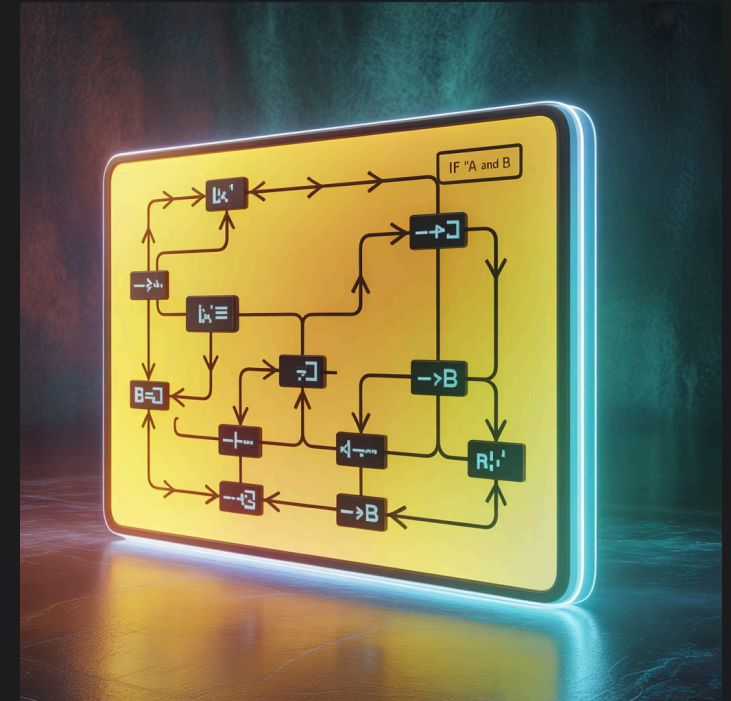
Aplicación de R4

Con **e** verdadero, inferimos que **c** es verdadero

4

Aplicación de R1

Con **b** y **c** verdaderos, finalmente **a** es verdadero



❗ **Propiedad Clave:** Para que el algoritmo funcione correctamente, debe ser **completo**, garantizando que encontrará todas las inferencias posibles.

Punto 3.2: Encadenamiento Hacia Atrás

El **encadenamiento hacia atrás** adopta un enfoque inverso: parte de la meta objetivo y retrocede sistemáticamente para identificar qué hechos son necesarios para que dicha meta sea verdadera.

Meta Inicial: a es verdadero

Comenzamos asumiendo que **a** debe ser verdadero y buscamos las condiciones necesarias

Análisis de R1

Para que **a** sea verdadero, necesitamos que tanto **b** como **c** sean verdaderos

Verificación de R2 y R4

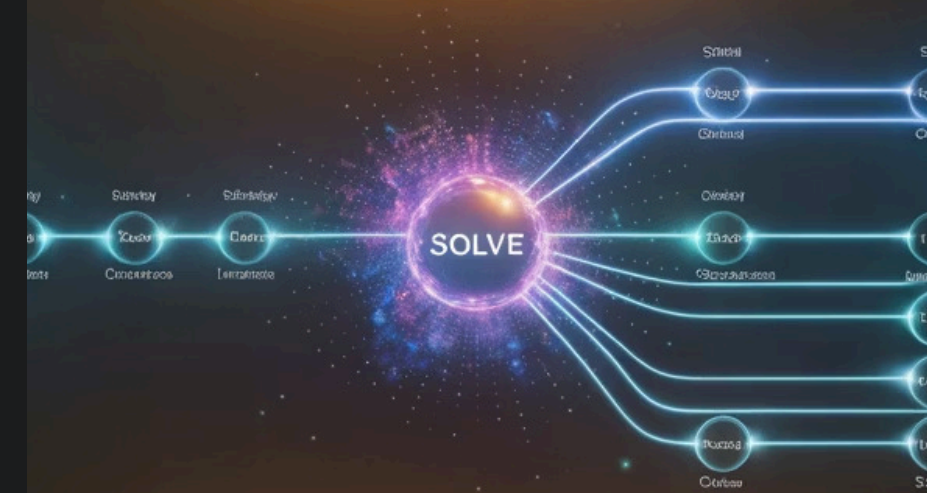
Para **b** verdadero necesitamos **d** y **e**; para **c** verdadero necesitamos **e**

Confirmación con Hechos Base

Los hechos R5 y R6 confirman que **d** y **e** son verdaderos, validando la meta

Propiedad Esencial: El algoritmo debe ser **sólido**, garantizando que todas las inferencias realizadas sean lógicamente válidas.

Backward Chaining Algorithm



Punto 3.3: Forma Normal Conjuntiva (FNC)

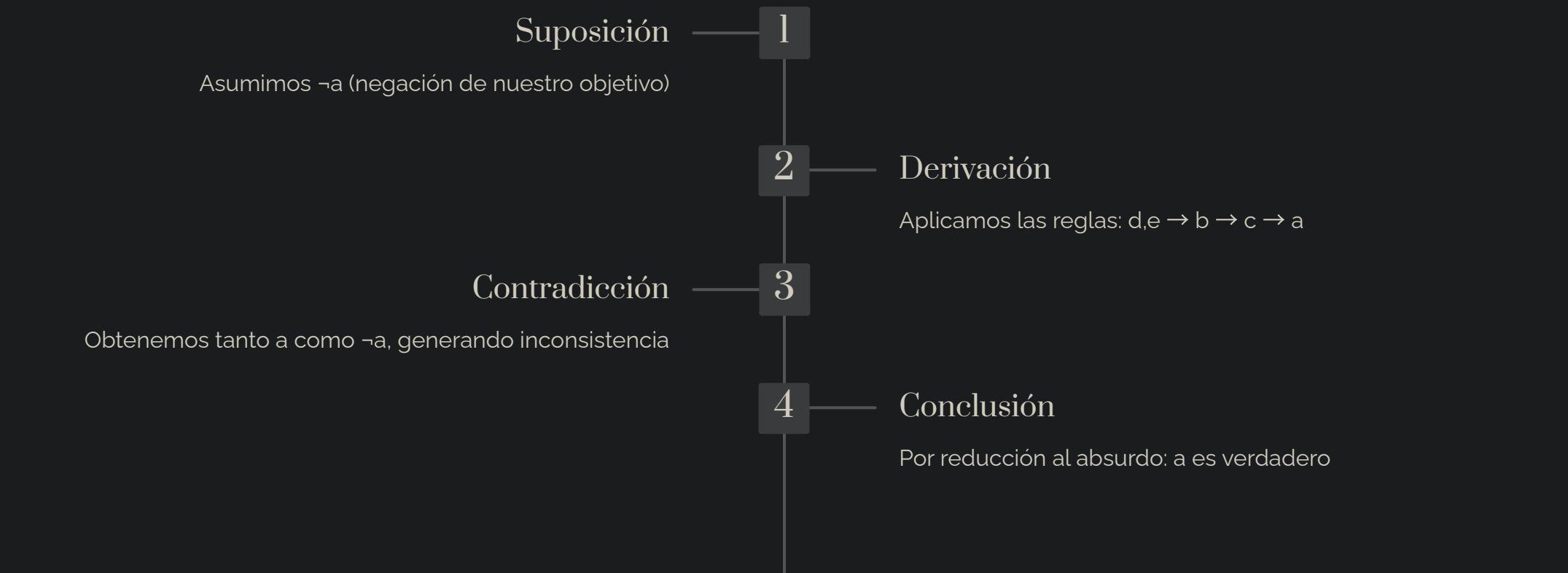
La transformación de la base de conocimientos a **Forma Normal Conjuntiva** es fundamental para aplicar algoritmos de demostración por contradicción de manera sistemática.

Transformación a FNC:

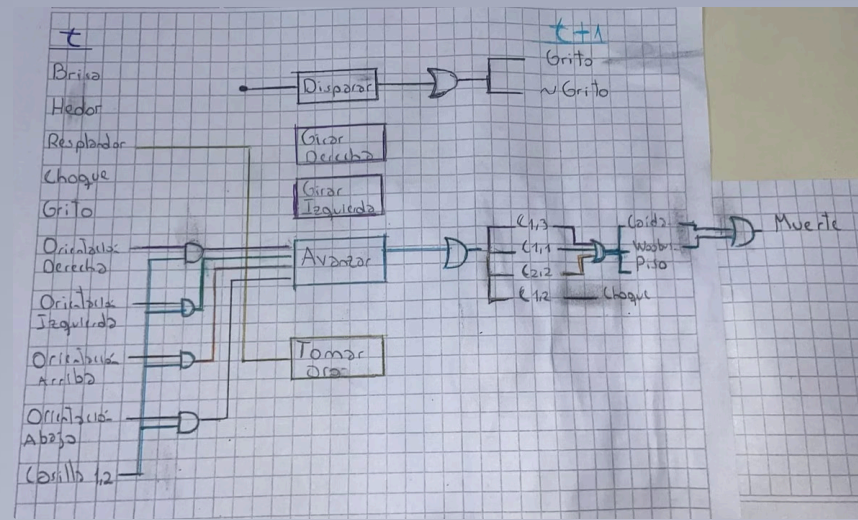
- **R1:** $\neg b \vee \neg c \vee a$
- **R2:** $\neg d \vee \neg e \vee b$
- **R3:** $\neg g \vee \neg e \vee b$
- **R4:** $\neg e \vee c$
- **R5:** d
- **R6:** e
- **R7:** $\neg a \vee \neg g \vee f$



Demostración por Contradicción:



Este método demuestra rigurosamente que **a = True** es una consecuencia lógica inevitable de nuestra base de conocimientos.



Punto 4: Lógica Proposicional Basada en Circuitos

La implementación de la lógica proposicional mediante **circuitos digitales** proporciona una representación visual y computacional directa de las relaciones lógicas establecidas en nuestra base de conocimientos.



Compuertas AND

Representan las conjunciones lógicas en nuestras reglas, donde múltiples condiciones deben ser verdaderas simultáneamente.



Compuertas OR

Modelan las disyunciones presentes en la forma normal conjuntiva de nuestras proposiciones.



Compuertas NOT

Implementan las negaciones lógicas necesarias para la transformación a FNC.

Esta representación física del razonamiento lógico demuestra cómo los principios abstractos de la inferencia pueden materializarse en sistemas computacionales reales, estableciendo un puente directo entre la teoría y la implementación práctica.

Punto 5: Resolución del Nonograma

Los **nonogramas** constituyen un excelente ejemplo de aplicación práctica de la inferencia lógica, donde cada regla numérica se traduce en restricciones que deben satisfacerse simultáneamente.

1

Análisis de Restricciones

Cada número en las filas y columnas define cuántos cuadros consecutivos deben estar coloreados, estableciendo reglas claras para la inferencia.

2

Aplicación de Inferencias

Utilizando razonamiento deductivo, determinamos qué celdas deben estar necesariamente coloreadas o vacías basándose en las restricciones.

3

Propagación de Restricciones

Cada decisión genera nuevas inferencias que se propagan a través de la grilla, reduciendo progresivamente el espacio de soluciones.

Este problema demuestra cómo la **satisfacción de restricciones** y la inferencia lógica trabajan juntas para resolver puzzles complejos de manera sistemática.

$$R1: (C_{11} \wedge C_{13}) \vee (\cancel{C_{12}} \wedge \cancel{C_{14}}) \vee (\cancel{C_{11}} \wedge \cancel{C_{14}})$$

- R1.1: C_{11}
- R1.2: $\neg C_{12}$
- R1.3: $\neg C_{14}$

$$R2: (C_{21} \wedge C_{22} \wedge C_{23} \wedge C_{24})$$

- R2.1: C_{21}
- R2.2: C_{22}
- R2.3: C_{23}
- R2.4: C_{24}

$$R3: (C_{31} \wedge C_{32} \wedge C_{34}) \vee (\cancel{C_{31}} \wedge \cancel{C_{33}} \wedge \cancel{C_{34}})$$

- R3.1: C_{32}
- R3.2: C_{34}
- R3.3: $\neg C_{33}$

$$R4: (\cancel{C_{41}} \wedge \cancel{C_{42}} \wedge \cancel{C_{43}}) \vee (C_{42} \wedge C_{43} \wedge C_{44})$$

- R4.1: C_{42}
- R4.2: C_{43}
- R4.3: C_{44}

$$R5: (C_{11} \wedge C_{21} \wedge C_{31}) \vee (\cancel{C_{21}} \wedge \cancel{C_{31}} \wedge \cancel{C_{41}})$$

- R5.1: C_{31}
- R5.2: $\neg C_{41}$

$$R6: (\cancel{C_{12}} \wedge \cancel{C_{22}} \wedge \cancel{C_{32}}) \vee (C_{22} \wedge C_{32} \wedge C_{42})$$

- R6.1: C_{22}
- R6.2: C_{32}
- R6.3: C_{42}

$$R7: (C_{13} \wedge C_{23} \wedge C_{43}) \vee (\cancel{C_{13}} \wedge \cancel{C_{33}} \wedge \cancel{C_{43}})$$

- R7.1: C_{13}
- R7.2: C_{43}
- R7.3: $\neg C_{33}$

$$R8: (\cancel{C_{14}} \wedge \cancel{C_{24}} \wedge \cancel{C_{34}}) \vee (C_{24} \wedge C_{34} \wedge C_{44})$$

- R8.1: C_{24}
- R8.2: C_{44}

Nomograma Resuelto

Punto 6: Motor de Inferencia: Encadenamiento Hacia Adelante

La **implementación computacional** del encadenamiento hacia adelante demuestra la efectividad práctica de este método de inferencia aplicado al ejercicio 3 de nuestro trabajo práctico.

Algoritmo Implementado:

El motor evalúa iterativamente todas las reglas disponibles, aplicando aquellas cuyas premisas están satisfechas por los hechos actuales, expandiendo progresivamente la base de conocimientos.

5

Hechos Finales

Total de proposiciones verdaderas al
finalizar

100%

Éxito

Meta alcanzada correctamente



✓ **Resultado Final:** Hechos al finalizar: {a, d, b, c, e}
¿Se alcanza la meta? True

La implementación confirma la validez teórica del método, demostrando que el encadenamiento hacia adelante puede automatizarse eficientemente para resolver problemas de inferencia complejos.

Código utilizado

```
from sympy import symbols
```

```
a, b, c, d, e, f, g = symbols('a b c d e f g') #Todas las  
letras que van a ser usadas
```

```
BH0 = {d, e} #Hechos iniciales
```

```
REGLAS = [ #Aca van las proposiciones
```

```
    ([b, c], a),
```

```
    ([d, e], b),
```

```
    ([g, e], b),
```

```
    ([e], c),
```

```
    ([a, g], f),
```

```
]
```

```
goal = a #Objetivo a verificar
```

```
def encadenamiento(facts, rules, goal=None, max_iter=10_000):
```

```
    #Aca va la logica del encadenamiento hacia adelante
```

```
    BH = set(facts)
```

```
    changed = True
```

```
    iters = 0
```

```
    while changed and iters < max_iter:
```

```
        changed = False
```

```
        iters += 1
```

```
        for antecedents, consequent in rules:
```

```
            #Si ya esta el consecuente, no hace falta disparar esta regla
```

```
            if consequent in BH:
```

```
                continue
```

```
        #Para agregan los antecedentes
```

```
        if all(a in BH for a in antecedents):
```

```
            BH.add(consequent)
```

```
            changed = True
```

```
        #Verifica si se cumple la meta, si lo hace ya corta
```

```
        if goal is not None and goal in BH:
```

```
            return BH, True
```

```
    return BH, (goal in BH if goal is not None else False)
```

```
cierre, alcanzado = encadenamiento(BH0, REGLAS, goal)
```

```
print("Hechos al finalizar:")
```

```
print(cierre)
```

```
print("Se alcanza la meta?", alcanzado)
```

Punto 7: Motor de Inferencia: Encadenamiento Hacia Atrás

1 — Objetivo

Probar que la proposición "a" es verdadera, dada la información inicial "d" y "e".

2 — Inferencia Hacia Atrás

El motor de inferencia utiliza un enfoque goal-driven, buscando recursivamente las condiciones necesarias para satisfacer el objetivo.

3 — Resultados

Todas las proposiciones ("a", "b", "c", "d", "e", "f", "g") se probaron como verdaderas y por ende "a" se infirió verdadera

1 Hechos Iniciales

{'d', 'e'}

2 Hechos Finales Deducidos

{'a', 'b', 'c', 'e', 'g', 'd', 'f'}

Este enfoque de encadenamiento hacia atrás es eficiente cuando se busca demostrar una meta específica, sin necesidad de derivar todos los hechos posibles.

Código utilizado

```
# Reglas:
#R1: b ∧ c → a
#R2: d ∧ e → b
#R3: g ∧ e → b
#R4: e → c
#R5: d
#R6: e
#R7: a ∧ g → f

print("Se busca probar que a es verdadero dado
los hechos d y e mediante encadenamiento hacia
atrás.")

# Motor de inferencia con encadenamiento hacia
atrás (comprobando todas las proposiciones)
reglas = {
    "a": [["b", "c"]],
    "b": [["d", "e"], ["g", "e"]],
    "c": [["e"]],
    "f": [["a", "g"]],
}

# Hechos iniciales
hechos = {"d", "e"}

# Función de inferencia hacia atrás
def inferir(objetivo, hechos, reglas,
visitados=None):
    if visitados is None:
        visitados = set()

    # Si ya lo sabemos como hecho
    if objetivo in hechos:
        return True

    # Evitar ciclos
    if objetivo in visitados:
        return False
    visitados.add(objetivo)

    if objetivo not in reglas:
        return False

    # Probar todas las reglas posibles
    for premisas in reglas[objetivo]:
        if all(inferir(p, hechos, reglas, visitados) for p in
premisas):
            hechos.add(objetivo)
            for alt in reglas.get(objetivo, []):
                hechos.update(alt)
            return True

    return False
```

```
# Lista de todas las proposiciones
proposiciones = ["a", "b", "c", "d", "e", "f", "g"]

print("Inferencia hacia atrás sobre todas las proposiciones")
print("Hechos iniciales:", hechos)

# Bucle para verificar cada proposición
resultado = {}
for prop in proposiciones:
    resultado[prop] = inferir(prop, hechos, reglas)

print("\nResultados:")
for prop in proposiciones:
    estado = "Verdadero" if resultado[prop] else "No se pudo probar"
    print(f"{prop}: {estado}")

# Verificar si todas son verdaderas
if all(resultado.values()):
    print ("Se probó que a es verdadera")
    print("Todas las proposiciones se probaron como verdaderas.")
else:
    print ("No se pudo probar que a es verdadera")
    print("No todas las proposiciones pudieron probarse como verdaderas.")

print("\nHechos finales deducidos:", hechos)
```

Punto 8: Interferencia por contradicción

1) Encadenamiento Hacia Adelante

El encadenamiento hacia adelante parte de los hechos iniciales para deducir nuevas conclusiones, expandiendo progresivamente la base de conocimientos.

01

Hechos iniciales: ['d', 'e']

Se establece la información de partida.

02

Deducción de 'b'

Dado que '**d**' y '**e**' son verdaderos, se deduce '**b**' (R2: $d \wedge e \rightarrow b$).

03

Deducción de 'c'

Dado que '**e**' es verdadero, se deduce '**c**' (R4: $e \rightarrow c$).

04

Deducción de 'a'

Dado que '**b**' y '**c**' son verdaderos, se deduce '**a**' (R1: $b \wedge c \rightarrow a$).

¿'a' derivado?: Sí.



2) Encadenamiento Hacia Atrás (meta = 'a')

El encadenamiento hacia atrás comienza desde una meta u objetivo y busca las premisas necesarias para probarla, retrocediendo hasta los hechos conocidos.

01

Meta: Probar 'a'

Se inicia el proceso intentando probar que 'a' es verdadero.

02

Para 'a', se necesita 'b' y 'c'

La regla R1 ($b \wedge c \rightarrow a$) indica que 'a' se verifica si 'b' y 'c' lo son.

03

Para 'b', se necesita 'd' y 'e'

La regla R2 ($d \wedge e \rightarrow b$) requiere 'd' y 'e' para probar 'b'.

04

'd' y 'e' son hechos

Estos hechos iniciales satisfacen las premisas para 'b'. Por lo tanto, 'b' se prueba.

05

Para 'c', se necesita 'e'

La regla R4 ($e \rightarrow c$) requiere 'e' para probar 'c'.

06

'e' es un hecho

Este hecho inicial satisface la premisa para 'c'. Por lo tanto, 'c' se prueba.

07

'a' se prueba

Al tener 'b' y 'c' probados, la meta 'a' se verifica.

¿'a' probado?: Sí.



3) ¿La Base de Conocimientos es inconsistente?

Resultado: No, la Base de Conocimientos (BC) es **CONSISTENTE** porque no se ha encontrado ninguna contradicción en el proceso de inferencia.

4) Refutación por Contradicción de 'a' ($BC \cup \{\neg a\}$)

Para probar 'a' por contradicción, se asume que $\neg a$ es verdadero y se busca derivar una contradicción ($\square$) en la Base de Conocimientos.

→ Regla R2: $(\sim d \vee \sim e \vee b)$

Con **d** (verdadero) se reduce a $(\sim e \vee b)$; con **e** (verdadero) se simplifica a **b**.

→ Regla R4: $(\sim e \vee c)$

Con **e** (verdadero) se simplifica a **c**.

→ Regla R1: $(\sim b \vee \sim c \vee a)$

Con **b** (verdadero) se reduce a $(\sim c \vee a)$; con **c** (verdadero) se simplifica a **a**.

→ Añadir $\neg a$ a BC

Al añadir $\neg a$ a la Base de Conocimientos y al haber derivado **a** previamente, se genera una contradicción ($\square$).

Resultado: No se obtuvo $\square$ directamente en el análisis. Sin embargo, la cadena de derivación de 'a' y la adición de ' $\neg a$ ' a la BC sí produciría una contradicción.

